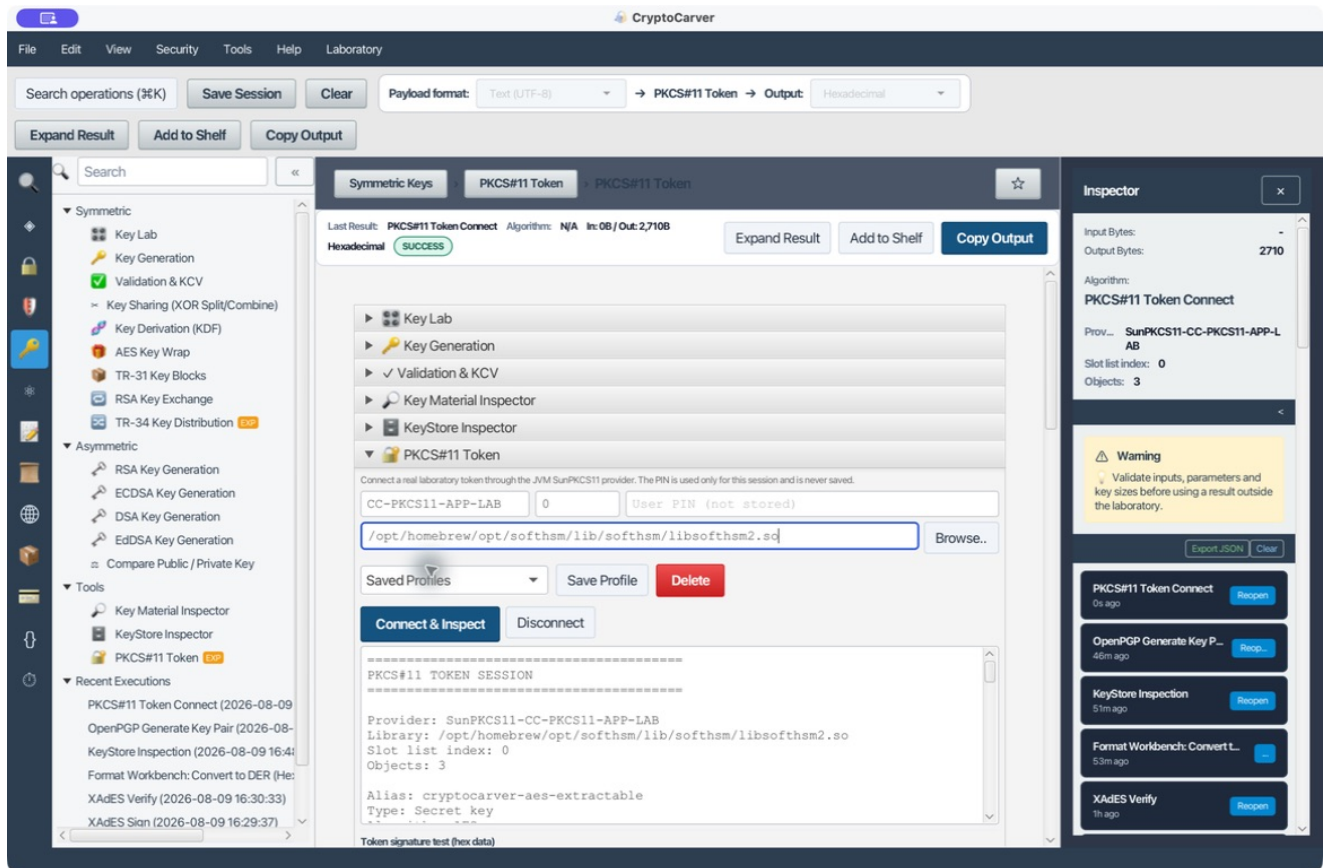


# Generación de un token PKCS#11 con SoftHSM y CryptoCarver



Este tutorial crea un token PKCS#11 de laboratorio, genera un par RSA dentro de él y lo conecta con CryptoCarver para demostrar una firma sin exportar la clave privada. SoftHSM simula la interfaz de un HSM; no sustituye los controles físicos, las ceremonias de claves ni la auditoría de producción.

CryptoCarver no inicializa tokens: opera un token que ya existe. La inicialización y la generación del objeto se realizan con la utilidad del proveedor; la conexión, inventario, firma, CMS, JWT y wrap/unwrap se realizan desde CryptoCarver.

## Qué se obtiene

| Artefacto           | Valor de laboratorio                   | Propiedad de seguridad                         |
|---------------------|----------------------------------------|------------------------------------------------|
| Token               | CC-PKCS11-APP-LAB                      | Tiene etiqueta y PIN de usuario propios.       |
| Slot lógico         | slotListIndex = 0                      | Es el índice que se configura en CryptoCarver. |
| Módulo PKCS#11      | libsofthsm2.so                         | Se carga por SunPKCS11.                        |
| Par RSA             | cc-app-signing, 2048 bits, CKA_ID = 02 | La privada nace en el token.                   |
| Operación de prueba | SHA256withRSA                          | La aplicación recibe una firma, no la privada. |

## Antes de empezar

En macOS con Homebrew, instala SoftHSM y OpenSC si todavía no existen: `brew install softhsm opensc`. Comprueba las herramientas con `softhsm2-util --version` y `pkcs11-tool --version`.

Para localizar el módulo, ejecuta `brew --prefix softhsm`. La ruta habitual de esta instalación es `/opt/homebrew/opt/softhsm/lib/softhsm/libsofthsm2.so`. En Linux suele ser una ruta terminada en `libsofthsm2.so`; en Windows, una DLL del proveedor. No copies la ruta de un equipo distinto: comprueba arquitectura y versión local.

Usa PINes exclusivos de laboratorio. En los ejemplos se emplean 87654321 como SO-PIN y 123456 como PIN de usuario. Sustitúyelos antes de cualquier uso fuera del laboratorio y no los incluyas en perfiles, capturas ni repositorios.

## Caso 1. Inicializar un token nuevo

Primero lista los slots disponibles: `softhsm2-util --show-slots`. Un slot libre presenta un token sin inicializar. Inicialízalo con la etiqueta y los PINes de laboratorio:

```
softhsm2-util --init-token --free --label CC-PKCS11-APP-LAB --so-pin 87654321 --pin 123456
```

Vuelve a ejecutar `softhsm2-util --show-slots`. Debes observar un token presente, inicializado, con User PIN init.: yes y la etiqueta CC-PKCS11-APP-LAB. SoftHSM puede reasignar un identificador numérico grande al slot; no confundas ese identificador con el `slotListIndex` usado por la aplicación.

Qué ha ocurrido. El SO-PIN administra el token y el PIN de usuario permite abrir una sesión para operar con objetos privados. Inicializar un token borra su contenido anterior; por eso se usa un slot libre para la práctica. En un HSM real, esta acción es una ceremonia controlada y no una tarea de aplicación rutinaria.

## Caso 2. Generar un par RSA dentro del token

Genera el par RSA residente, seleccionando el primer slot de la lista y autenticando con el PIN de usuario:

```
pkcs11-tool --module /opt/homebrew/opt/softhsm/lib/softhsm/libsofthsm2.so --slot-index 0 --login --pin 123456 --keypairgen --key-type rsa:2048 --id 02 --label cc-app-signing --usage-sign
```

Para inventariar objetos: `pkcs11-tool --module /opt/homebrew/opt/softsm/lib/softsm/libsoftsm2.so --slot-index 0 --login --pin 123456 --list-objects`.

Salida esperada. Deben aparecer una clave privada y una pública RSA, ambas con la etiqueta `cc-app-signing` y ID: 2 (0x02). La privada debe informar `sensitive, always sensitive, never extractable` y `local`. Esa es la evidencia de que se generó localmente en el token y no fue importada como bytes desde la JVM.

| Atributo                | Debe permitir                            | Debe impedir o limitar                              |
|-------------------------|------------------------------------------|-----------------------------------------------------|
| CKA_SIGN                | Firmar con la privada                    | Firmar si el objeto solo es de cifrado.             |
| CKA_DECRYPT             | Descifrado RSA si la política lo permite | Usar una clave de firma para descifrar por defecto. |
| CKA_EXTRACTABLE = false | Mantener la clave dentro del token       | Exportar la privada o envolverla para extraerla.    |
| CKA_ID                  | Vincular pública, privada y certificado  | Depender solo de un alias que pueda cambiar.        |

## Caso 3. Conectar e inventariar desde CryptoCarver

- 1 Abre Claves → PKCS#11 Token.
- 2 En `*Token profile name*`, escribe `CC-PKCS11-APP-LAB`.
- 3 En `*Slot index*`, escribe 0.
- 4 En `*User PIN*`, usa únicamente el PIN de la sesión de laboratorio.
- 5 En `*Native PKCS#11 library*`, selecciona la ruta de `libsoftsm2`.
- 6 Pulsa `Connect & Inspect`.

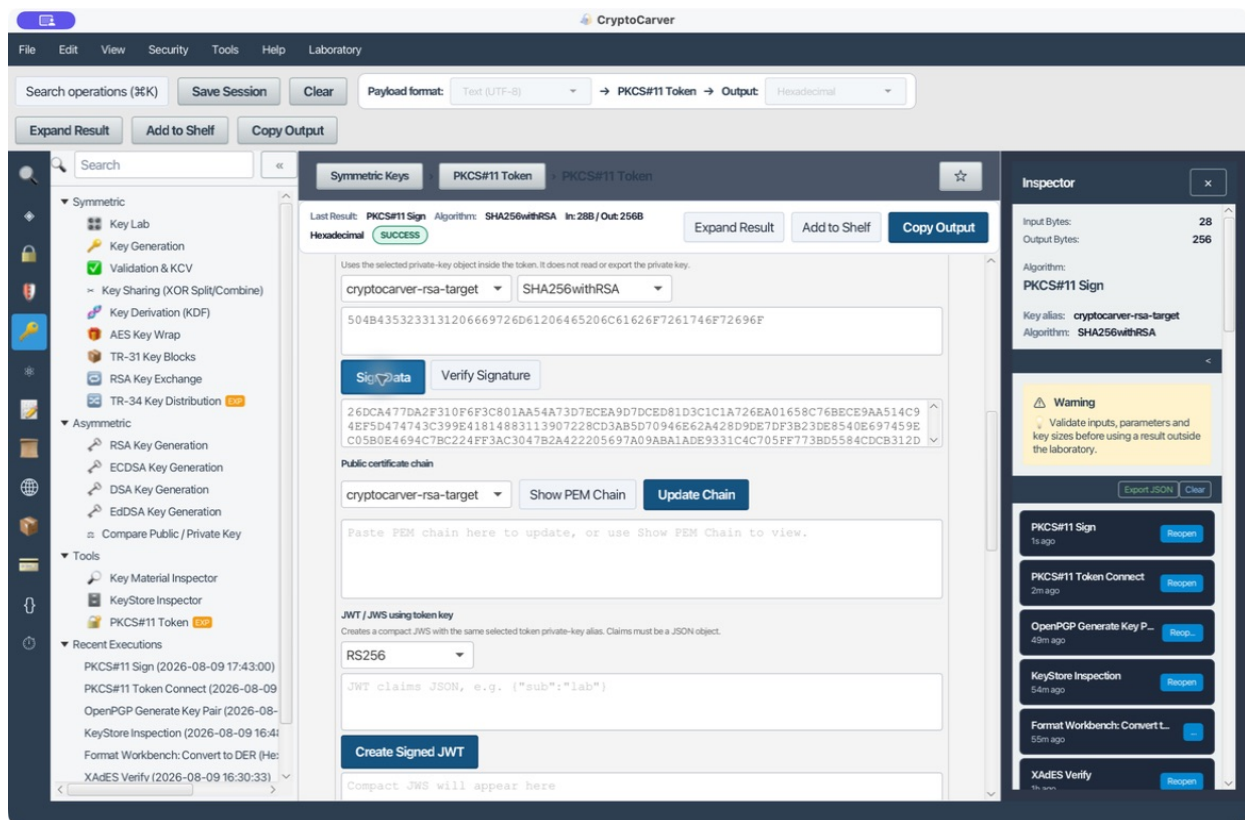
La ejecución reproducida conecta el proveedor `SunPKCS11-CC-PKCS11-APP-LAB`, muestra el slot y el número de objetos, y expone solamente metadatos. Las privadas residentes se anuncian como `Token-resident` y su huella como `Not exported`.

Lectura correcta del informe. La sección de servicios JCA informa de compatibilidad del proveedor, no de una garantía por objeto. Antes de usar `SHA256withRSA`, `AES-GCM` o `RSA-OAEP`, comprueba que el objeto tiene el atributo correspondiente y que el mecanismo está soportado por el token.

## Caso 4. Firmar y verificar sin exportar la privada

- 1 Selecciona el alias privado ofrecido por el token y `SHA256withRSA`.
- 2 Introduce el mensaje hexadecimal  
`504B4353233131206669726D61206465206C61626F7261746F72696F`.
- 3 Pulsa `Sign Data`. Son los 28 bytes de PKCS#11 firma de laboratorio.
- 4 Pulsa `Verify Signature` sin modificar ni entrada ni firma.

Resultado reproducido. CryptoCarver generó una firma RSA de 256 bytes. El resultado muestra el alias y el algoritmo; la clave privada nunca se presenta como PEM, DER ni hexadecimal.



Caso 4: firma con clave privada residente

Prueba negativa. Cambia solo el último byte de entrada de 6F a 6E y verifica de nuevo. Debe fallar. No firmes el mensaje modificado: se está comprobando la integridad de la firma original.

## Caso 5. Reutilizar el token en los demás flujos

| Necesidad             | Operativa de CryptoCarver      | Entrada                                                             | Evidencia correcta                                          |
|-----------------------|--------------------------------|---------------------------------------------------------------------|-------------------------------------------------------------|
| Emitir un JWS         | Create Signed JWT              | {"sub":"laboratorio","scope":"firma","iat":1760000000}              | JWS compacto verificable con la pública del token.          |
| Firmar CMS            | Create CMS SignedData          | Contenido hexadecimal; marca *Detached* cuando el contrato lo exija | CMS Base64 y validación posterior con el inspector CMS.     |
| Publicar cadena       | Show PEM Chain                 | Alias de certificado                                                | PEM público y huellas; no claves privadas.                  |
| Solicitar certificado | CSR con fuente PKCS#11         | Sujeto y alias activo                                               | La CSR demuestra posesión mediante firma de token.          |
| Firmar XML/PDF/ASiC   | Fuente Connected PKCS#11 Token | Documento y política correspondiente                                | Firma válida y cadena de confianza comprobada.              |
| Transportar una clave | Wrap / Unwrap key objects      | Clave envolvente, objeto destino y mecanismo                        | Blob envuelto o resumen de unwrap; nunca la clave en claro. |

CMS detached exige guardar exactamente el contenido original, porque no queda embebido. Para JWT, verificar la firma no basta: también valida iss, aud, fechas y la lista permitida de algoritmos.

## Diagnóstico y recuperación segura

| Síntoma                                         | Causa habitual                                  | Corrección                                                     |
|-------------------------------------------------|-------------------------------------------------|----------------------------------------------------------------|
| Connect & Inspect falla al cargar la biblioteca | Ruta, arquitectura o dependencias incorrectas   | Usa la ruta absoluta de la biblioteca del sistema actual.      |
| Token equivocado                                | Se usó ID numérico en lugar del índice de lista | Confirma etiqueta y usa slotListIndex.                         |
| Login fallido                                   | PIN de usuario incorrecto o token bloqueado     | Verifica estado del token; no pruebes PINes de producción.     |
| No aparece el alias                             | CKA_ID, etiqueta, clase u objetos de otro slot  | Inventaría con pkcs11-tool y compara identificadores.          |
| La firma falla                                  | Falta CKA_SIGN o mecanismo compatible           | Revisa atributos y mecanismo, no exportes la clave.            |
| Wrap rechazado                                  | Clave objetivo no extraíble                     | Es la política correcta; crea un flujo de transporte aprobado. |

## Cierre del laboratorio

Pulsa Disconnect al terminar: la sesión PKCS#11 pertenece al proceso de CryptoCarver y el PIN no se guarda. Conserva la etiqueta del token, el módulo, el índice de slot y los CKA\_ID en la documentación de laboratorio; elimina el token de pruebas cuando ya no sea necesario mediante el procedimiento administrado de SoftHSM. No ejecutes --init-token sobre un token con datos que quieras preservar.